

Encoding, Escala e Vazamento de Dados

Onde a maioria dos números que parecem bons demais nasce

Transformar categorias em números e colocar variáveis na mesma escala parecem tarefas mecânicas — até que se descobre que a forma errada de fazer as duas coisas é a origem mais comum de vazamento de dados: informação do futuro (ou do conjunto de teste) contaminando o treino. Este material cobre as famílias de encoding e escalonamento, e formaliza, com Pipeline, a disciplina que impede o vazamento de acontecer por acidente.

Nível: Intermediário

Pre-requisitos: Feature Engineering (módulo deste tema); Vetores, Matrizes e Projeções (tema 2)

Carga sugerida: 8 a 10 horas (leitura + 3 notebooks)

Notebooks: 01-encoding-de-categoricas · 02-escalonamento · 03-vazamento-de-dados · 99-exercicios

Autoria: Material do curso de ML & DL

Versao: 1.0

Sumário

1. Por que este módulo existe	3
1.1 O que você vai conseguir fazer ao final	3
2. Encoding de variáveis categóricas	4
2.1 One-hot: o padrão para nominal, sem ordem	4
2.2 Ordinal: quando a ordem é real	4
2.3 Frequência: uma alternativa simples para alta cardinalidade	4
2.4 Target encoding: poderoso, e o mais fácil de vazar	4
3. Escalonamento	6
3.1 As três famílias	6
3.2 Quando escalonamento importa (e quando não importa)	6
4. Vazamento de dados: taxonomia completa	7
4.1 `Pipeline` e `ColumnTransformer`: a solução estrutural	7
5. Erros que custam caro — checklist	8
6. Para ir além	9

1. Por que este módulo existe

Um número que parece bom demais quase sempre é. A causa mais comum de "meu modelo teve 98% de acurácia na validação e 60% em produção" não é um algoritmo ruim — é vazamento de dados: alguma etapa de preparação, aplicada antes ou de forma incorreta em relação ao split treino/teste, deixou o modelo "ver" no treino informação que só existiria no futuro. Encoding de categóricas e escalonamento de variáveis são, coincidentemente, as duas etapas onde esse erro mais acontece — porque parecem transformações inócuas demais para merecer cuidado.

ANALOGIA

Treinar um modelo e testá-lo com dados vazados é como estudar para uma prova depois de ver o gabarito. A nota vai ser ótima — e não vai dizer nada sobre o quanto você realmente aprendeu. O gabarito, aqui, é qualquer estatística (uma média, uma frequência, um mínimo/máximo) calculada usando dados que, no mundo real, o modelo não teria acesso no momento da previsão.

1.1 O que você vai conseguir fazer ao final

- Escolher entre one-hot, ordinal, frequência e target encoding com justificativa técnica.
 - Aplicar target encoding sem vaziar a média do próprio alvo para dentro da feature.
 - Escolher o escalonador certo (`StandardScaler`, `MinMaxScaler`, `RobustScaler`) considerando a presença de outliers.
 - Construir um `Pipeline/ColumnTransformer` que torna o vazamento estruturalmente impossível, em vez de depender de disciplina manual.
-

2. Encoding de variáveis categóricas

2.1 One-hot: o padrão para nominal, sem ordem

Já visto no tema 2: cria uma coluna binária por categoria. Correto para variáveis **nominais** (sem ordem natural), mas cresce linearmente com a cardinalidade — 500 categorias viram 500 colunas.

ARMADILHA COMUM

One-hot completo (sem remover uma categoria) junto com intercepto cria dependência linear exata (tema 2) em modelos lineares. Em árvores e gradient boosting isso não quebra nada matematicamente, mas ainda desperdiça uma coluna. `drop="first"` (ou equivalente) é a prática padrão para modelos lineares; para árvores, tanto faz.

2.2 Ordinal: quando a ordem é real

Para categorias com ordem natural ("**básico**" < "**intermediário**" < "**avançado**"), codificar como inteiros sequenciais preserva a ordem sem explodir dimensionalidade. O erro contrário — usar ordinal em uma variável sem ordem real ("**Norte**", "**Sul**", "**Sudeste**") — inventa uma relação de distância que não existe: o modelo passa a achar "Sul" mais "parecido" com "Sudeste" do que com "Norte" só pela numeração escolhida, sem nenhuma base real.

2.3 Frequência: uma alternativa simples para alta cardinalidade

Substituir cada categoria pela sua frequência (ou contagem) no dataset. Barato, não aumenta dimensionalidade, e captura "quão comum" a categoria é — útil quando isso já carrega sinal (categorias raras costumam se comportar diferente das comuns).

2.4 Target encoding: poderoso, e o mais fácil de vazar

Substitui cada categoria pela **média do alvo** dentro daquela categoria. Poderosíssimo para alta cardinalidade (não explode dimensionalidade, e captura diretamente a relação com o alvo) — e a técnica de encoding mais fácil de implementar errado.

ARMADILHA COMUM

Calcular a média do alvo por categoria usando **a linha atual incluída no cálculo** vaza o próprio valor do alvo daquela linha para dentro da feature — na prática, a feature "aprende de cor" o alvo de cada linha em categorias raras (uma categoria com 1 única observação teria a feature igual ao próprio alvo). Isso não é um vazamento sutil: é usar a resposta como pergunta.

FORMULARIO

A correção padrão é o **target encoding com validação cruzada (out-of-fold)**: divida o treino em k folds; para calcular o encoding das linhas do fold i , use **apenas** a média do alvo calculada nos outros $k - 1$ folds. Isso garante que nenhuma linha "veja" seu próprio valor de alvo refletido na própria feature. `sklearn.preprocessing.TargetEncoder` já implementa essa lógica internamente.

Uma segunda proteção comum é a **suavização** (shrinkage): para categorias raras, misturar a média da categoria com a média global, ponderada pelo número de observações — evitando que uma categoria com poucos exemplos receba um encoding extremo e ruidoso.

3. Escalonamento

3.1 As três famílias

Escalonador	Fórmula	Sensível a outliers?	Quando usar
<code>StandardScaler</code>	$(x - \bar{x})/s$	Sim — média e desvio são sensíveis	Padrão geral, dados aproximadamente simétricos
<code>MinMaxScaler</code>	$(x - \min)/(max - \min)$	Muito — um único outlier redefine a escala inteira	Quando se precisa de um intervalo fixo $[0, 1]$ (ex.: redes neurais com certas ativações)
<code>RobustScaler</code>	$(x - \text{mediana})/IQR$	Não — usa estatísticas robustas	Dados com outliers conhecidos, já discutido no módulo 2

NO MERCADO

A escolha do escalonador não é estética — ela decide o quanto outliers dominam o resultado. Um único valor extremo faz o `MinMaxScaler` espremer 99% dos dados "normais" em uma faixa minúscula perto de 0, porque a escala inteira é ancorada no mínimo e no máximo. `RobustScaler`, usando mediana e IQR (tema 1), ignora esse efeito.

3.2 Quando escalonamento importa (e quando não importa)

Retomando o tema 2: modelos que dependem de distância (k-NN, k-means, SVM) ou de gradiente descendente (regressão linear/logística, redes neurais) exigem padronização — sem ela, features de escalas diferentes distorcem a geometria do problema e o condicionamento da otimização. **Árvores de decisão e seus ensembles (Random Forest, gradient boosting) são invariantes a escalonamento monotônico** — o particionamento em um único ponto de corte por vez não é afetado por reescalar uma variável.

ARMADILHA COMUM

Escalonar não custa nada mesmo quando desnecessário (árvores), então "sempre escalonar por segurança" parece atalho seguro — exceto que escalonar **destrói a interpretabilidade direta** de um coeficiente e não é gratuito quando aplicado a variáveis categóricas já codificadas como 0/1 (escalonar uma dummy raramente faz sentido). Aplique escalonamento de forma intencional, coluna a coluna, não em bloco automático.

4. Vazamento de dados: taxonomia completa

Os módulos anteriores já tocaram três formas específicas. Aqui está o quadro completo:

DEFINIÇÃO

Vazamento de pré-processamento: ajustar (fit) qualquer transformador — imputador, encoder, escalonador — usando dados que incluem o conjunto de teste, antes da divisão. Visto nos módulos 2 e 3.

Vazamento de alvo (target leakage): uma feature que é, na prática, uma proxy ou derivação do próprio alvo — calculada usando informação que só existiria depois do evento que se quer prever.

Vazamento temporal: usar informação futura (relativa ao ponto sendo previsto) em qualquer feature de série temporal ou histórico. Visto no módulo 3.

Vazamento de grupo: múltiplas linhas da mesma entidade (mesmo cliente, mesmo paciente, mesma máquina) distribuídas entre treino e teste. O modelo "memoriza" padrões específicos daquela entidade no treino e parece performar bem no teste — mas só porque já viu (uma versão d)a mesma entidade antes.

NO MERCADO

Vazamento de grupo é sutil e comum em dados médicos e de sensores IoT: várias medições do mesmo paciente/equipamento, divididas aleatoriamente entre treino e teste, parecem observações independentes mas não são. A correção é o `GroupKFold` (particionar por grupo, nunca por linha) — tema 6 aprofunda os esquemas corretos de validação cruzada para esses casos.

4.1 Pipeline e ColumnTransformer: a solução estrutural

A forma mais confiável de eliminar vazamento de pré-processamento não é lembrar de fazer certo toda vez — é usar uma ferramenta que torna o erro estruturalmente difícil de cometer. `Pipeline` encadeia transformação → modelo em um único objeto; `cross_val_score` e `GridSearchCV` aplicados a um `Pipeline` automaticamente ajustam cada transformador **apenas nos dados de treino de cada fold**, e aplicam a transformação (já ajustada) aos dados de validação daquele fold.

FORMULÁRIO

A regra prática: se uma etapa de preparação depende de estatística calculada a partir dos dados (média, desvio-padrão, frequência de categoria, média do alvo por categoria), ela **precisa** estar dentro do `Pipeline`, nunca aplicada ao dataset inteiro antes do split ou da validação cruzada.

5. Erros que custam caro — checklist

- Ajustar `StandardScaler`, `OneHotEncoder` ou qualquer imputador no dataset completo antes do split treino/teste.
- Calcular target encoding sem esquema out-of-fold, vazando a própria linha para sua própria feature.
- Usar `MinMaxScaler` em dados com outliers conhecidos sem antes tratá-los (módulo 2) ou trocar por `RobustScaler`.
- Escalonar variáveis dummy (0/1) sem necessidade.
- Dividir treino/teste por linha quando várias linhas pertencem à mesma entidade (vazamento de grupo).
- Não usar `Pipeline/ColumnTransformer`, deixando a prevenção de vazamento depender só de lembrar a ordem certa manualmente.

6. Para ir além

- Documentação do `sklearn.compose.ColumnTransformer` e `sklearn.pipeline.Pipeline` — a implementação de referência para o fluxo correto.
- Kaufman et al., *Leakage in Data Mining: Formulation, Detection, and Avoidance* — o artigo mais citado sobre taxonomia de vazamento.
- Micci-Barreca (2001), o artigo original de target encoding com suavização bayesiana.